Forecasts at every level of the hierarchy (research note, 2026-10-22)

NO CODE CHANGED. Research only. The held-out run (22 arms, seeds 1-10) is flying on the frozen commit c2dfc42 and must not be touched; everything below comes after it, on development seeds.

The idea

Each level outputs a short prediction of what should happen next, alongside its normal output: - the planner forecasts each step's expected outcome, e.g. "after the gap: pillar ahead, about 6 m"; - the VLM forecasts the target's distance and side; - the VLA forecasts the next box and the remaining motion. D207 already does this: its line is `{"bbox_2d_next": [...]}` followed by `Left +x,+y,+z,+yaw` (native_line in plugins/inference/qwen_actions.py; logged in every C7/C8 flight).

There are two uses: 1. **Measurement.** Score each level's precision (how often its forecast comes true) separately from task success. This is paper conclusion 2: how good the LLM, VLM and VLA each are at planning and predicting. 2. **A handle (H10).** When a forecast and the measured outcome disagree, trigger a replan or recovery immediately, instead of waiting for a timeout.

Verdict: a good idea, with one hard rule

The literature supports both uses, and it names the one way to get this wrong:

Finding	Source	Consequence for us
LLM writes per-skill constraints, a VLM checks them during execution, a violation replans at once: higher success, shorter time	DoReMi 2023	Use 2 is a published, working pattern
Constraints as compact geometric elements checked by code in real time, both reactive and proactive: +28.7% success under disturbance	Code-as-Monitor, CVPR 2025	Forecasts must be measurable quantities (distance, side, visible, opening), checked by code
Self-critique by the same LLM collapses accuracy (16% to 2%); an external verifier raises it (38%)	Stechly et al., ICLR 2025; Valmeekam 2023	**Hard rule: never let the model check its own forecast.** The range sensor, depth or geometry checks it
Verbalised confidence is overconfident; consistency is better	Xiong et al., ICLR 2024	Ask for a checkable number, not "how sure are you"
A single success rate hides which module fails; score per module	Embodied Agent Interface, NeurIPS 2024	The per-level scoring frame for conclusion 2
A VLN agent that predicts its own progress gains 8 points	Ma et al., ICLR 2019	Predicting progress helps navigation, not only measurement
The VLA predicts a subgoal image before acting: +17%	CoT-VLA, CVPR 2025	VLA-level forecasting helps; ours forecasts a box, which is cheaper

The rule fits what we already measured: H5 routing worked because the sensor answered instead of the VLM (D-175), and H9 worked because depth replaced the VLA's written distance (D-182). A forecast-check handle is the same principle, applied over time: the model predicts and the sensor judges.

What the field tried and dropped, or what does not fit here

- **Self-verification** (the model checks its own plan or forecast): it fails in planning. Ruled out.
- **Verbalised confidence**: overconfident. Ruled out; use numbers that can be checked.
- **Image world models** (Pathdreamer, Dreamwalker, CoT-VLA's subgoal images): they imagine future views. Pathdreamer's look-ahead gives about half the gain of real views, and they are heavy. They are out of scope: our latency floor (D-177) already blocks time-critical tasks with a 4B model, so forecasts must be a few extra tokens in calls we already make, never a new call.

Options, ranked by effort against expected effect

#	Option	Effort	Expected effect	Recommendation
1	Score each level from the existing logs, using evaluator truth offline only: planner program correctness against the mission; VLM box -> world position error and hit rate; VLA box IoU and written-distance error (e.g. 2.6 m for 12 m)	Low: no runtime change, no GPU, runs during the held-out flights	Paper conclusion 2 with numbers, on all 432 + handle-run flights	**Do first**
2	Forecast fields in calls we already make (planner step `expect: {visible, distance_m, side}`, VLM `distance_m`), logged and scored against the sensor afterwards; a switch, off by default	Medium: prompt and contract change, dev-seed runs	Per-level forecast precision; tells us whether 4B forecasts are good enough to act on	Second, after the held-out run
3	H10 forecast-mismatch handle (DoReMi / Code-as-Monitor style): at step end the executive checks the forecast against measured facts, and a mismatch triggers recovery	Medium-high	Unknown; it could help closing_passage (expected "opening ahead", measured "none") and the gate. A noisy 4B forecast would cause false replans	Only if option 2 shows forecasts are precise enough

The order matters: option 2 measures whether the forecasts are good before option 3 acts on them. That avoids building a handle on top of predictions that turn out to be wrong.

Out of scope

- Any change to the frozen held-out run.
- Training the planner or VLM to forecast. These are prompt-level forecasts on the same frozen model, matching "one model, one runtime".
- Evaluator truth at run time. Truth is used only offline, to score.

Sources

- DoReMi: https://arxiv.org/abs/2307.00329
- Code-as-Monitor: https://arxiv.org/abs/2412.04455
- Stechly et al., self-verification limitations: https://arxiv.org/abs/2402.08115
- Valmeekam et al., self-critiquing plans: https://arxiv.org/abs/2310.08118
- Xiong et al., LLM uncertainty: https://arxiv.org/abs/2306.13063
- KnowNo: https://arxiv.org/abs/2307.01928
- Embodied Agent Interface: https://arxiv.org/abs/2410.07166
- LLM-DP: https://arxiv.org/abs/2308.06391
- GVL: https://arxiv.org/abs/2411.04549

- VLM success detectors: https://arxiv.org/abs/2303.07280
- Self-monitoring VLN agent: https://arxiv.org/abs/1901.03035
- REFLECT: https://arxiv.org/abs/2306.15724
- CoT-VLA: https://arxiv.org/abs/2503.22020
- Dreamwalker: https://arxiv.org/abs/2308.07498
- Pathdreamer: https://arxiv.org/abs/2105.08756

PDFs: papers/forecasts/ (README lists the mechanism taken from each).